

// Jivops

Flannel

Guía completa — 30 lecciones

Fundamentos · Redes overlay y backends · Instalación · Operación de red ·
Troubleshooting · Producción avanzada

Contenido

Módulo 1: Fundamentos de redes de contenedores y Flannel

- Lección 01 — El problema de networking que resuelve Flannel en Kubernetes
- Lección 02 — Qué es CNI y dónde encaja Flannel
- Lección 03 — Modelo de red de Kubernetes — requisitos que Flannel debe cumplir
- Lección 04 — Arquitectura de Flannel — flanneld y el subnet lease
- Lección 05 — Flannel vs otros plugins CNI (Calico, Cilium, Weave)

Módulo 2: Redes overlay y backends de Flannel

- Lección 06 — Qué es una red overlay y por qué Flannel la usa
- Lección 07 — Backend VXLAN — el backend por defecto
- Lección 08 — Backend host-gw — rendimiento sin encapsulación
- Lección 09 — Backend UDP — el backend original (y por qué evitarlo)
- Lección 10 — Elegir el backend apropiado según el entorno

Módulo 3: Instalación y configuración

- Lección 11 — Instalación de Flannel en un cluster de Kubernetes
- Lección 12 — El ConfigMap de configuración de Flannel
- Lección 13 — Asignación de subredes por nodo
- Lección 14 — Integración de Flannel con etcd o el almacén de Kubernetes
- Lección 15 — Flannel con kubeadm — configuración típica

Módulo 4: Operación de la red con Flannel

- Lección 16 — Cómo viaja un paquete entre Pods de distintos nodos con VXLAN
- Lección 17 — MTU y fragmentación de paquetes con overlay networking
- Lección 18 — Rutas y la tabla de rutas que Flannel configura
- Lección 19 — Flanneld — logs y diagnóstico del daemon
- Lección 20 — Interacción de Flannel con iptables y kube-proxy

Módulo 5: Troubleshooting y limitaciones

- Lección 21 — Diagnóstico de conectividad Pod-a-Pod fallida con Flannel
- Lección 22 — Problemas comunes de MTU y su solución
- Lección 23 — Qué NO resuelve Flannel — ausencia de NetworkPolicy
- Lección 24 — Combinar Flannel con Calico solo para NetworkPolicy (Canal)
- Lección 25 — Limitaciones de escalabilidad de Flannel en clusters grandes

Módulo 6: Producción avanzada

Lección 26 — Seguridad de red con Flannel — qué cubre y qué no

Lección 27 — Monitoreo de la red de Flannel

Lección 28 — Migración desde Flannel hacia otro CNI

Lección 29 — Cuándo Flannel es la elección correcta para producción

Lección 30 — Proyecto final: red de cluster con Flannel lista para producción

Módulo 1: Fundamentos de redes de contenedores y Flannel

Lección 01 — El problema de networking que resuelve Flannel en Kubernetes

Permite que los Pods ubicados en distintos nodos del cluster puedan comunicarse entre sí usando sus direcciones IP de Pod, asignando a cada nodo una subred única y gestionando el encaminamiento del tráfico entre esas subredes distribuidas en distintas máquinas.

Sin un plugin de red coordinando las subredes entre nodos, cada nodo manejaría su propio rango de direcciones de forma aislada, sin ninguna ruta configurada que le indique cómo alcanzar las direcciones de Pods que viven físicamente en una máquina distinta; Flannel es precisamente lo que coordina esa asignación de subredes entre nodos y configura las rutas necesarias para que el tráfico pueda viajar correctamente entre ellos.

Lección 02 — Qué es CNI y dónde encaja Flannel

CNI es una especificación estándar que define cómo un runtime de contenedores debe configurar la red de un Pod al crearlo, y Flannel es una implementación concreta de ese estándar, siendo uno de varios plugins CNI distintos que pueden usarse para cumplir esa especificación.

Como Flannel implementa la misma interfaz estandarizada de CNI que siguen otros plugins compatibles, Kubernetes y el runtime de contenedores interactúan con la capa de red a través de esa interfaz común, sin depender de detalles específicos y propietarios de Flannel; esa adherencia al estándar es precisamente lo que permite cambiar hacia otro plugin CNI compatible distinto sin tener que modificar cómo Kubernetes o el runtime interactúan con la red subyacente.

Lección 03 — Modelo de red de Kubernetes — requisitos que Flannel debe cumplir

Cada Pod debe poder comunicarse con cualquier otro Pod del cluster sin necesitar traducción de direcciones (NAT) entre ellos, es decir, cada Pod ve y usa la IP real del otro Pod directamente, sin importar en qué nodo distinto esté ubicado cada uno.

Sin NAT, la aplicación dentro de un Pod simplemente usa la dirección IP real del Pod destino para conectarse, sin tener que lidiar con traducciones de direcciones que podrían variar según el contexto o la ubicación; ese modelo simplificado, donde cada Pod tiene una identidad de red directa y consistente, facilita significativamente razonar sobre cómo las aplicaciones se conectan entre sí, comparado con tener que gestionar capas adicionales de traducción compleja.

Lección 04 — Arquitectura de Flannel — flanneld y el subnet lease

flanneld solicita y mantiene un subnet lease único para su nodo (un rango de direcciones IP reservado exclusivamente para los Pods de ese nodo específico), y configura las rutas e interfaces de red necesarias para que el tráfico hacia esa subred funcione correctamente dentro del esquema general del cluster.

Si todos los nodos compartieran el mismo rango de direcciones sin ninguna exclusividad real, dos Pods ubicados en nodos físicamente distintos podrían terminar con exactamente la misma dirección IP asignada, causando un conflicto real de conectividad; el subnet lease único y exclusivo por nodo garantiza que cada rango de direcciones esté reservado para un único nodo específico, evitando por completo ese escenario de direcciones duplicadas entre Pods de distintos nodos.

Lección 05 — Flannel vs otros plugins CNI (Calico, Cilium, Weave)

Flannel se enfoca principalmente en resolver la conectividad básica entre Pods de distintos nodos de forma simple y confiable, sin incluir por defecto capacidades más avanzadas como la aplicación de políticas de red (NetworkPolicy), que sí están integradas de forma nativa en plugins como Calico o Cilium.

Si la necesidad real del equipo es únicamente conectividad básica confiable entre Pods, sin requerir las capacidades adicionales de políticas de red avanzadas que un plugin como Cilium ofrece, la simplicidad de Flannel evita la sobrecarga de aprender y mantener funcionalidades adicionales que ese equipo específico no necesitaría usar, una elección pragmática basada en los requisitos reales del proyecto en vez de elegir automáticamente el plugin con más funcionalidades disponibles.

Módulo 2: Redes overlay y backends de Flannel

Lección 06 — Qué es una red overlay y por qué Flannel la usa

Una red virtual construida encima de la red física subyacente, donde el tráfico entre Pods se encapsula dentro de paquetes que viajan por la red física existente, permitiendo que los Pods se comuniquen como si estuvieran en la misma red local aunque físicamente estén en nodos distintos conectados por una red más compleja.

Como el tráfico de la red overlay se encapsula dentro de paquetes estándar que la red física subyacente ya sabe manejar sin ningún requisito especial, esa red física (sin importar si es de un proveedor cloud específico, on-premise, o cualquier otro entorno) simplemente transporta esos paquetes encapsulados normalmente; esa abstracción es precisamente lo que permite que Flannel funcione de forma consistente sobre infraestructuras de red físicas muy distintas entre sí, sin requerir una configuración especial previa en cada una.

Lección 07 — Backend VXLAN — el backend por defecto

Encapsula los paquetes originales dentro de paquetes UDP usando el protocolo VXLAN, que añade una cabecera adicional identificando la red virtual correspondiente, permitiendo que el tráfico viaje a través de la red física existente aunque esta no tenga conocimiento directo de las direcciones IP de los Pods.

Como el tráfico de los Pods queda encapsulado dentro de un paquete UDP estándar, la red física subyacente solo necesita saber transportar ese paquete UDP normal de un punto a otro, sin necesitar ningún conocimiento especial sobre las direcciones IP de los Pods o la topología específica de Kubernetes; esa encapsulación es precisamente lo que permite que VXLAN funcione sobre cualquier red física genérica compatible con IP estándar, sin requisitos especiales adicionales relacionados con Kubernetes.

Lección 08 — Backend host-gw — rendimiento sin encapsulación

host-gw ofrece mejor rendimiento al evitar el overhead de encapsular y desencapsular cada paquete, a cambio de requerir que todos los nodos del cluster estén en la misma red de capa 2 (mismo segmento de red local), una limitación que VXLAN no tiene gracias a su encapsulación.

Cada paquete que viaja a través de VXLAN requiere un trabajo adicional real de encapsularlo en el origen y desencapsularlo en el destino, además de que el paquete resultante es más grande que el original por la cabecera añadida; host-gw evita por completo ese paso adicional de procesamiento usando rutas directas en su lugar, lo que se traduce en un mejor rendimiento de red cuando la limitación de requerir la misma red de capa 2 es aceptable para ese entorno específico.

Lección 09 — Backend UDP — el backend original (y por qué evitarlo)

El backend UDP realiza la encapsulación completamente en espacio de usuario (en vez de aprovechar soporte de encapsulación VXLAN disponible directamente en el kernel del sistema operativo), resultando en un rendimiento significativamente más bajo comparado con VXLAN, que sí aprovecha esa aceleración disponible en el kernel.

Procesar la encapsulación en espacio de usuario requiere transferir los datos de cada paquete entre el kernel del sistema operativo y el programa que corre en espacio de usuario, una transferencia adicional que introduce overhead real de rendimiento en cada paquete individual procesado; VXLAN, al tener soporte de encapsulación integrado directamente en el kernel, evita esa transferencia adicional, procesando el trabajo de forma mucho más directa y eficiente sin ese paso intermedio.

Lección 10 — Elegir el backend apropiado según el entorno

Evaluar si todos los nodos del cluster están garantizados de estar siempre en la misma red de capa 2 (permitiendo usar host-gw para mejor rendimiento), o si esa garantía no se cumple de forma confiable (requiriendo la flexibilidad de VXLAN, que funciona independientemente de esa topología de red específica).

En un cluster desplegado across múltiples zonas de disponibilidad distintas, no hay garantía real de que todos los nodos estén conectados dentro de la misma red de capa 2 local, un requisito específico que host-gw necesita cumplir para funcionar correctamente; VXLAN, al no depender de esa misma restricción de topología de red, funciona de forma confiable independientemente de cómo estén distribuidos geográficamente los nodos entre esas distintas zonas.

Módulo 3: Instalación y configuración

Lección 11 — Instalación de Flannel en un cluster de Kubernetes

Aplicar el manifiesto YAML oficial de Flannel (que incluye un DaemonSet para que flanneld corra en cada nodo, los permisos RBAC necesarios, y un ConfigMap con la configuración), confirmando que el rango de direcciones de Pods configurado coincida con el usado al inicializar el cluster.

Kubernetes ya tiene definido un rango específico de direcciones de Pods desde el momento en que se inicializó el cluster (por ejemplo, con kubeadm); si Flannel se configurara con un rango distinto a ese, existiría una inconsistencia real entre lo que Kubernetes espera y lo que Flannel está realmente asignando, pudiendo causar que los Pods no reciban direcciones correctas o que la red del cluster falle de forma confusa y difícil de diagnosticar.

Lección 12 — El ConfigMap de configuración de Flannel

El rango total de direcciones de red que Flannel gestiona para todo el cluster, y la configuración del backend específico a usar (como VXLAN o host-gw), información que cada flanneld necesita para coordinarse correctamente con los demás nodos del mismo cluster.

Distintos backends (VXLAN, host-gw) usan mecanismos de encapsulación o enrutamiento incompatibles entre sí; si un nodo usara VXLAN mientras otro usara host-gw, ninguno de los dos podría interpretar correctamente el tráfico que el otro está enviando con un mecanismo distinto, rompiendo la comunicación entre ellos; el ConfigMap compartido garantiza que todos los nodos del cluster usen consistentemente el mismo backend configurado.

Lección 13 — Asignación de subredes por nodo

flanneld en el nuevo nodo solicita una subred disponible dentro del rango total configurado para el cluster, consultando y registrando esa asignación en el almacén de datos compartido (típicamente etcd o el propio almacén de Kubernetes), evitando que dos nodos distintos terminen con subredes que se solapen entre sí.

El almacén de datos compartido da visibilidad real a todos los nodos sobre qué subredes ya están asignadas y cuáles siguen disponibles; sin esa coordinación centralizada, si cada nodo decidiera su propia subred de forma completamente aislada e independiente, dos nodos distintos podrían coincidentalmente elegir exactamente la misma subred sin saberlo, mientras consultar el almacén compartido antes de asignar permite verificar y reservar una subred genuinamente disponible, evitando ese conflicto real de solapamiento.

Lección 14 — Integración de Flannel con etcd o el almacén de Kubernetes

Flannel puede almacenar su estado directamente en el almacén de datos de Kubernetes (usando recursos personalizados o anotaciones de nodo, el enfoque más común en instalaciones modernas) o, alternativamente, conectarse a un cluster de etcd propio separado específicamente para Flannel, un enfoque usado en configuraciones más antiguas o independientes de Kubernetes.

Desplegar un cluster de etcd completamente separado solo para Flannel requeriría operar y mantener otra pieza adicional de infraestructura distinta, con su propio proceso de backup, monitoreo y actualización separado; aprovechar el almacén de datos que Kubernetes ya usa internamente evita esa duplicación, simplificando la operación general para un equipo que ya tiene experiencia manteniendo ese almacén existente, sin necesitar aprender a gestionar un sistema adicional completamente independiente.

Lección 15 — Flannel con kubeadm — configuración típica

El rango de direcciones (CIDR) para los Pods del cluster, especificado con la opción `--pod-network-cidr` de `kubeadm`, que debe coincidir exactamente con el rango que luego se configure en el manifiesto de instalación de Flannel.

Kubeadm configura internamente a Kubernetes para esperar un rango específico de direcciones de Pods según ese parámetro (o su valor por defecto si se omite); si Flannel se configura después con un rango distinto al que Kubernetes realmente espera, existiría una inconsistencia real entre ambos componentes que podría no manifestarse como un error visible durante la instalación de Flannel en sí, pero sí causaría fallos reales y confusos más adelante al intentar asignar direcciones a los Pods.

Módulo 4: Operación de la red con Flannel

Lección 16 — Cómo viaja un paquete entre Pods de distintos nodos con VXLAN

El paquete original sale del Pod, es interceptado y encapsulado por flanneld en el nodo A dentro de un paquete VXLAN/UDP, viaja a través de la red física real hasta el nodo B, donde flanneld lo recibe, lo desencapsula, y lo entrega al Pod destino usando su dirección IP real original.

El Pod destino espera recibir el paquete en su formato original normal, exactamente como el Pod de origen lo envió antes de ser encapsulado para el transporte por la red física; si se entregara el paquete todavía con la capa adicional de encapsulación VXLAN sin remover, la aplicación dentro del Pod no sabría interpretar ni procesar correctamente ese formato adicional, por lo que flanneld debe desencapsularlo primero, restaurando el paquete a su forma original antes de entregarlo.

Lección 17 — MTU y fragmentación de paquetes con overlay networking

La encapsulación VXLAN añade una cabecera adicional a cada paquete, reduciendo el espacio real disponible para los datos originales; si el MTU dentro del Pod no se ajusta para dejar espacio para esa cabecera adicional, los paquetes podrían fragmentarse innecesariamente o incluso descartarse, degradando el rendimiento de la red.

Si el paquete original ya usa el MTU máximo que la red física permite, sin dejar ningún margen adicional disponible, añadir la cabecera extra de encapsulación VXLAN haría que el paquete final resultante excediera ese límite físico real de la red; ese paquete demasiado grande tendría que fragmentarse en partes más pequeñas (con el overhead adicional que eso implica) o simplemente sería descartado, razón por la cual ajustar el MTU dentro del Pod dejando margen para esa cabecera adicional es importante.

Lección 18 — Rutas y la tabla de rutas que Flannel configura

Añade entradas de ruta que indican que el tráfico destinado hacia la subred de otro nodo específico debe enviarse a través de la interfaz de red overlay correspondiente (como la interfaz VXLAN creada por flanneld), en vez de intentar enviarse directamente como si estuviera en la red local.

Sin esa entrada de ruta específica, el sistema operativo trataría cualquier dirección de destino como si estuviera en la red local normal, intentando resolverla directamente sin pasar por ninguna encapsulación especial; como la dirección del Pod destino realmente vive en otro nodo físico distinto, ese intento fallaría, mientras la entrada de ruta configurada por Flannel le indica explícitamente al sistema que ese tráfico específico debe pasar por la interfaz de red overlay para poder encapsularse y viajar correctamente hacia el nodo correcto.

Lección 19 — Flanneld — logs y diagnóstico del daemon

Mensajes sobre el proceso de obtención del subnet lease para ese nodo, confirmación de la configuración del backend aplicada, y posibles errores de conectividad al intentar comunicarse con el almacén de datos compartido (Kubernetes o etcd) usado para la coordinación entre nodos.

Los logs de flanneld en ese nodo específico revelarían directamente si el daemon logró obtener correctamente su subnet lease, si la configuración del backend se aplicó sin errores, y si hay algún problema de conectividad con el almacén de datos compartido; revisar esa fuente directa y específica del componente responsable de la red en ese nodo es un punto de partida lógico y eficiente antes de investigar causas más generales o menos dirigidas del problema de conectividad reportado.

Lección 20 — Interacción de Flannel con iptables y kube-proxy

Son capas complementarias y distintas: Flannel provee la conectividad de red subyacente entre Pods, mientras kube-proxy (usando iptables u otro mecanismo) implementa la funcionalidad de Services, traduciendo una IP virtual de Service hacia la IP real de un Pod específico, aprovechando la conectividad que Flannel ya hizo posible.

Kube-proxy traduce la IP virtual de un Service hacia la IP real de un Pod específico, pero esa traducción por sí sola no sirve de nada si no existe una forma real de alcanzar físicamente esa dirección de Pod, especialmente si está en otro nodo distinto; sin que Flannel haya resuelto primero esa conectividad básica subyacente entre nodos, la traducción de kube-proxy apuntaría hacia un destino técnicamente correcto pero prácticamente inalcanzable, dejando la funcionalidad de Service sin efecto real.

Módulo 5: Troubleshooting y limitaciones

Lección 21 — Diagnóstico de conectividad Pod-a-Pod fallida con Flannel

Verificar primero que los Pods de flanneld estén corriendo correctamente en ambos nodos involucrados, revisar sus logs en busca de errores, confirmar que las rutas estén configuradas correctamente en ambos nodos, y verificar que no haya un firewall o regla de iptables bloqueando el tráfico de la red overlay específicamente.

Si flanneld no está corriendo correctamente en alguno de los nodos involucrados, ninguna otra configuración más detallada (rutas, firewall) importaría realmente, ya que el componente central responsable de toda la conectividad de red overlay simplemente no estaría funcionando; verificar ese aspecto más básico y fundamental primero es un enfoque sistemático y eficiente que evita invertir tiempo investigando causas más complejas antes de descartar el problema más evidente y probable.

Lección 22 — Problemas comunes de MTU y su solución

Conexiones que funcionan correctamente para peticiones pequeñas (como un simple ping) pero fallan o se vuelven extremadamente lentas al transferir datos más grandes (como descargar un archivo), un patrón característico de paquetes que exceden el MTU real disponible y terminan fragmentándose o descartándose.

Un ping típicamente envía paquetes pequeños que probablemente siguen estando dentro del límite real disponible incluso después de añadir la cabecera adicional de encapsulación VXLAN; una transferencia de archivo grande, en cambio, envía paquetes más grandes que tienen mucha más probabilidad de exceder ese límite real una vez encapsulados, causando fragmentación o descarte específicamente para ese tráfico de paquetes grandes, mientras los paquetes pequeños del ping nunca llegan a presentar ese mismo problema.

Lección 23 — Qué NO resuelve Flannel — ausencia de NetworkPolicy

La aplicación de NetworkPolicy, el recurso nativo de Kubernetes que permite restringir qué tráfico de red está permitido entre Pods según reglas específicas; Flannel por sí solo provee conectividad pero no implementa ni aplica esas políticas de restricción de tráfico.

Flannel resuelve específicamente el problema de conectividad básica entre Pods, sin incluir ningún mecanismo real que filtre o restrinja ese tráfico según reglas específicas; sin un componente adicional (como Calico) que implemente y aplique activamente las reglas de NetworkPolicy definidas, el comportamiento por defecto del cluster sería permitir conectividad completamente abierta entre cualquier par de Pods, sin ninguna restricción real aplicándose sobre ese tráfico.

Lección 24 — Combinar Flannel con Calico solo para NetworkPolicy (Canal)

Usa Flannel para resolver la conectividad básica de red (su fortaleza principal), mientras aprovecha específicamente el componente de aplicación de políticas de Calico para añadir soporte de NetworkPolicy, combinando lo mejor de ambos proyectos sin necesitar adoptar la solución de networking completa de Calico.

El equipo ya tiene experiencia real y conocimiento acumulado operando específicamente Flannel para resolver la conectividad básica del cluster; migrar por completo hacia Calico requeriría aprender y adoptar un sistema de networking completamente nuevo y distinto desde cero, mientras una solución híbrida como Canal permite añadir únicamente la capacidad específica de NetworkPolicy que falta, aprovechando el

conocimiento y la infraestructura ya existente sin ese costo de una migración completa innecesaria.

Lección 25 — Limitaciones de escalabilidad de Flannel en clusters grandes

El mecanismo de coordinación de subnet leases y la propagación de información de rutas entre todos los nodos puede volverse más costoso de mantener sincronizado a medida que el número de nodos crece significativamente, un desafío que plugins más avanzados pueden abordar con mecanismos más sofisticados de distribución de esa información.

Cada nodo adicional añadido al cluster significa más información de subred que coordinar y más participantes que mantener sincronizados entre sí sobre el estado completo de esas asignaciones; un mecanismo de coordinación que funciona eficientemente con decenas de nodos puede volverse significativamente más costoso de mantener sincronizado cuando hay miles de nodos participando simultáneamente, un desafío real de escala que se vuelve mucho más notorio en clusters muy grandes.

Módulo 6: Producción avanzada

Lección 26 — Seguridad de red con Flannel — qué cubre y qué no

Flannel asegura que el tráfico de Pods pueda viajar correctamente encapsulado a través de la red física entre nodos, pero no cifra ese tráfico por defecto ni restringe qué tráfico está permitido según políticas de seguridad, dejando esas responsabilidades adicionales a otras herramientas complementarias si se necesitan.

La instalación estándar de Flannel no incluye cifrado automático del tráfico encapsulado entre nodos sin una configuración adicional específica; si el requisito real de la organización es que todo el tráfico interno esté cifrado, necesitaría cerrar esa brecha añadiendo una capa adicional (ya sea una configuración específica de cifrado o una herramienta complementaria), ya que la instalación por defecto de Flannel no cubre ese aspecto de seguridad sin esa adición explícita.

Lección 27 — Monitoreo de la red de Flannel

Métricas sobre errores de encapsulación o desencapsulación de paquetes, latencia de la red overlay entre nodos, y el estado de las subredes asignadas, dando visibilidad real de si la red overlay está funcionando eficientemente más allá de solo confirmar que el proceso está técnicamente activo.

El estado 'Running' solo confirma que el proceso de flanneld está técnicamente activo y no se ha caído, pero no dice nada sobre si ese proceso está funcionando correctamente en la práctica; podría estar activo mientras experimenta errores reales de encapsulación o una latencia elevada en la red overlay, problemas que solo métricas específicas de comportamiento (no solo el estado básico del proceso) podrían revelar y permitir diagnosticar a tiempo.

Lección 28 — Migración desde Flannel hacia otro CNI

Planificar una estrategia que minimice la interrupción del tráfico durante la transición (ya que los Pods existentes seguirían usando el mecanismo de red anterior hasta ser recreados), considerando probablemente reiniciar los Pods de forma gradual para que adopten el nuevo plugin sin causar una interrupción completa simultánea del cluster.

Si todos los Pods se reiniciaran simultáneamente para adoptar el nuevo plugin CNI, el cluster completo quedaría sin ninguna instancia activa funcionando durante ese período de transición, causando una interrupción total del servicio; una estrategia gradual, en cambio, mantendría algunos Pods con el mecanismo anterior todavía funcionando mientras otros van adoptando el nuevo plugin de forma escalonada, evitando esa interrupción completa simultánea.

Lección 29 — Cuándo Flannel es la elección correcta para producción

Cuando la necesidad real es conectividad básica y confiable entre Pods, sin requerir NetworkPolicy avanzado ni capacidades de service mesh integradas, priorizando la simplicidad operativa y una curva de aprendizaje más suave sobre el conjunto más amplio de funcionalidades que ofrecen plugins más complejos.

Si la necesidad real del equipo en ese entorno de producción específico es únicamente conectividad confiable entre Pods, sin requisitos avanzados de políticas de seguridad de red, la simplicidad operativa de Flannel sigue siendo completamente apropiada incluso en producción; estar en producción no significa automáticamente necesitar el plugin más avanzado y complejo disponible, sino aquel que mejor se ajusta a los requisitos reales y específicos de ese proyecto particular.

Lección 30 — Proyecto final: red de cluster con Flannel lista para producción

Backend apropiado elegido según la topología real de red (VXLAN u host-gw) + MTU correctamente ajustado + integración verificada con kubeadm/Kubernetes + monitoreo de métricas de la red overlay + plan claro sobre NetworkPolicy (combinando con Calico si es necesario) + procedimiento de diagnóstico documentado.

Intenta mencionar al menos dos pilares: elección apropiada de backend según la topología real, MTU correctamente ajustado, monitoreo de la red overlay o un plan claro sobre NetworkPolicy.